



Métodos Numéricos Computacionais

Igor dos Reis Gomes

RELATÓRIO TRABALHO 3 – SISTEMAS DE EQUAÇÕES LINEARES

Métodos Numéricos Computacionais

Bauru

2025

Bauru
2025
Igor dos Reis Gomes

RELATÓRIO TRABALHO 3 – SISTEMAS DE EQUAÇÕES LINEARES

Métodos Numéricos Computacionais

Trabalho teórico-prático apresentado
como parte da composição da nota da
disciplina de Métodos Numéricos
Computacionais – Prof. Douglas
Rodrigues.

1. OBJETIVO

O objetivo deste trabalho é aplicar diferentes métodos numéricos para resolver um sistema linear que representa o equilíbrio de forças em uma estrutura mecânica com o uso de três barras. Foram implementados métodos diretos, como a Eliminação de Gauss-Compacto, Gauss-Jordan, Decomposição LU e Fatoração de Cholesky, e métodos iterativos, como Jacobi-Richardson e Gauss-Seidel. Além de realizar a comparação entre tempo de execução, número de iterações e erro relativo de todos os métodos que foram aplicados.

2. EXPLICAÇÃO DO CÓDIGO

Este trabalho foi desenvolvido usando a linguagem de programação Python, com o auxílio de bibliotecas para construção do projeto, as bibliotecas utilizadas foram: os, time, numpy, matplotlib e tabulate. O sistema é composto por dois arquivos principais: `igor_dos_reis_gomes_t3.py`, que contém a interface de usuário e as funções para gerar gráficos e tabelas, e `metodos.py`, que contém a implementação de todos os métodos numéricos e as funções de verificação de critérios das matrizes. Para os métodos que são diretos, foram implementadas funções que resolvem o sistema de uma só vez, e para os métodos iterativos, foram iniciados com uma aproximação dada e calculam sucessivas melhorias para a solução final. O código utiliza um erro de no máximo de 10^{-5} e máximo de 50 iterações.

2.1. Decomposição LU

A função `metodoDecomposicaoLU()` fatora a matriz dos coeficientes A em duas matrizes, uma triangular inferior (L) e uma triangular superior (U). O sistema $Ax = b$ é então transformado em dois sistemas mais simples: $Ly = b$ e $Ux = y$. Para verificação da possibilidade de aplicação do método, é realizada a análise dos menores principais da matriz. No trecho abaixo é mostrada a implementação desse método.

Trecho 1 - Implementação da Decomposição LU

```
1 # Método de Decomposição LU
2 def metodoDecomposicaoLU(matriz):
3     if not (quadrada(matriz) and menoresPrincipais(matriz)):
4         return False, None, None
5
6     n = len(matriz)
7     L = np.eye(n)
8     U = np.zeros((n, n))
9
10    for i in range(n):
11        for j in range(i, n):
12            U[i, j] = matriz[i, j] - sum(L[i, k] * U[k, j] for k in range(i))
13        for j in range(i + 1, n):
14            L[j, i] = (matriz[j, i] - sum(L[j, k] * U[k, i] for k in range(i))) / U[i, i]
15
16    return True, L, U
```

Fonte: Elaboração Própria

2.2. Eliminação de Gauss-Compacto

A função `metodoGaussCompacto()` aplica a eliminação de Gauss para triangular a matriz *A* e resolve o sistema usando substituição regressiva, ou seja, transforma a matriz *A* em uma matriz triangular superior *U*, e armazenando os multiplicadores na matriz *L*. O sistema ampliado é formado e modificado linha a linha, eliminando as variáveis abaixo da diagonal. No final, obtém-se a solução a partir da última equação para a primeira. A função verifica se a matriz é quadrada e se seus pivôs são não nulos antes de prosseguir.

Trecho 2 - Implementação da Eliminação de Gauss-Compacto

```
1 def metodoGaussCompacto(matriz):
2     if not (quadrada(matriz) and temPivos(matriz) and temInversa(matriz)):
3         return False, None, None
4
5     n = len(matriz)
6     L = np.eye(n)
7     U = matriz.copy().astype(float)
8
9     for i in range(n):
10        for j in range(i + 1, n):
11            fator = U[j, i] / U[i, i]
12            L[j, i] = fator
13            U[j, i:] -= fator * U[i, i:]
14
15    return True, L, U
```

Fonte: Elaboração Própria

2.3. Eliminação de Gauss-Jordan

A função `metodoGaussJordan()` transforma a matriz dos coeficientes A em uma matriz identidade. A função zera os elementos acima e abaixo da diagonal principal do sistema, resultando em uma matriz identidade. A solução final está diretamente no vetor estendida, após as operações. É verificado se a matriz é quadrada, se seus pivôs são não nulos e se possui inversa antes de prosseguir.

Trecho 3 - Implementação da Eliminação de Gauss-Jordan

```
1  # Método de Gauss-Jordan
2  def metodoGaussJordan(matriz, termos):
3      if not (quadrada(matriz) and temPivos(matriz) and temInversa(matriz)):
4          return False, None
5
6      n = len(matriz)
7      termos = termos.reshape(-1, 1)
8      estendida = np.hstack((matriz.astype(float), termos.astype(float)))
9
10     for i in range(n):
11         estendida[i] = estendida[i] / estendida[i, i]
12         for j in range(n):
13             if i != j:
14                 estendida[j] -= estendida[j, i] * estendida[i]
15
16     return True, estendida[:, -1]
```

Fonte: Elaboração Própria

2.4. Fatoração de Cholesky

A função `metodoCholesky()` implementa a decomposição de Cholesky, que representa a matriz A como LL^T , onde L é uma matriz triangular inferior. Esse método é mais eficiente e estável, mas só pode ser aplicado se A for simétrica e definida positiva, condições verificadas inicialmente na função. A solução é obtida em duas etapas semelhantes ao método LU.

Trecho 4 - Implementação da Fatoração de Cholesky

```
1 # Método de Cholesky
2 def metodoCholesky(matriz):
3     if not (simetrica(matriz) and menoresPrincipais(matriz, positivos=True)):
4         return False, None
5
6     n = len(matriz)
7     L = np.zeros((n, n))
8
9     for i in range(n):
10         for j in range(i + 1):
11             soma = sum(L[i, k] * L[j, k] for k in range(j))
12             if i == j:
13                 L[i, j] = math.sqrt(matriz[i, i] - soma)
14             else:
15                 L[i, j] = (matriz[i, j] - soma) / L[j, j]
16
17     return True, L
```

Fonte: Elaboração Própria

2.5. Método de Jacobi-Richardson

A função `metodoJacobi()` resolve o sistema de forma iterativa, atualizando todos os valores de x simultaneamente em cada iteração com base nos valores da iteração anterior. O processo se repete até que o erro entre iterações fique abaixo de uma tolerância definida. A função também armazena o histórico de cada valor de x_i a cada iteração, o que permite gerar posteriormente, gráficos de convergência. A convergência é garantida se a matriz for diagonalmente dominante, critério que é verificado pela função.

Trecho 5 - Implementação do método de Jacobi-Richardson

```
1 # Método de Jacobi
2 def metodoJacobi(matriz, termos):
3     if not (diagonalDominanteLinhas(matriz) and diagonalDominanteColunas(matriz)):
4         return False, None, None, None
5
6     n = len(matriz)
7     max_iter = 50
8     tol = 1e-5
9     x = np.zeros(n)
10    historico = []
11
12    for _ in range(max_iter):
13        novo_x = np.zeros(n)
14        for i in range(n):
15            soma = sum(matriz[i, j] * x[j] for j in range(n) if j != i)
16            novo_x[i] = (termos[i] - soma) / matriz[i, i]
17
18        erro = np.linalg.norm(novo_x - x, ord=np.inf) / np.linalg.norm(novo_x, ord=np.inf)
19        x = novo_x
20        historico.append(x.copy())
21
22        if erro < tol:
23            return True, x, len(historico), np.array(historico), erro
24
25    erro = np.linalg.norm(novo_x - x, ord=np.inf) / np.linalg.norm(novo_x, ord=np.inf)
26    return False, x, max_iter, np.array(historico), erro
```

Fonte: Elaboração Própria

2.6. Método de Gauss-Seidel

A função `metodoGaussJordan()` é muito similar com o método de Jacobi, porém utiliza os valores recém-calculados das componentes de x na mesma iteração, o que geralmente acelera a convergência. O código verifica a convergência pelo critério de Sassenfeld e pelo critério da diagonal dominante. Assim como no Jacobi, o histórico das iterações é salvo para geração de gráficos para posterior análise.

Trecho 6 - Implementação do método de Gauss-Seidel

```
1 def metodoGaussSeidel(matriz, termos):
2     if not (diagonalDominanteLinhas(matriz) and criterioSassenfeld(matriz)):
3         return False, None, None, None
4
5     n = len(matriz)
6     max_iter = 50
7     tol = 1e-5
8     x = np.zeros(n)
9     historico = []
10
11     for _ in range(max_iter):
12         x_ant = x.copy()
13         for i in range(n):
14             soma1 = sum(matriz[i, j] * x[j] for j in range(i))
15             soma2 = sum(matriz[i, j] * x_ant[j] for j in range(i + 1, n))
16             x[i] = (termos[i] - soma1 - soma2) / matriz[i, i]
17
18         erro = np.linalg.norm(x - x_ant, ord=np.inf) / np.linalg.norm(x, ord=np.inf)
19         historico.append(x.copy())
20
21         if erro < tol:
22             return True, x, len(historico), np.array(historico), erro
23
24     erro = np.linalg.norm(x - x_ant, ord=np.inf) / np.linalg.norm(x, ord=np.inf)
25     return False, x, max_iter, np.array(historico), erro
```

Fonte: Elaboração Própria

2.7. Solução dos Sistemas Lineares

Na função `solucao()`, é resolvido o sistema linear $Ax = b$ após a matriz A ter sido decomposta em duas matrizes triangulares: L e U , como ocorre nos métodos de decomposição LU e fatoração de Cholesky. Primeiro, é resolvido o sistema intermediário $Ly = b$ usando substituição direta, linha a linha. Em seguida, resolve-se $Ux = y$ com substituição reversa, de trás para frente. Ambas as etapas utilizam o método tradicional de substituição, aproveitando as propriedades das matrizes triangulares para obter a solução x . O parâmetro `ok` permite pular o cálculo caso a decomposição falhe. Abaixo, pode-se verificar a implementação:

Trecho 7 - Implementação da solução dos Sistemas Lineares

```
1  # SOLUÇÃO DOS SISTEMAS LINEARES
2  def solucao(L, U, b, ok=True):
3      if not ok:
4          return None
5      n = len(b)
6
7      # Substituição direta para  $Ly = b$ 
8      y = np.zeros(n)
9      for i in range(n):
10         y[i] = b[i] - sum(L[i][j] * y[j] for j in range(i))
11
12     # Substituição reversa para  $Ux = y$ 
13     x = np.zeros(n)
14     for i in reversed(range(n)):
15         x[i] = (y[i] - sum(U[i][j] * x[j] for j in range(i + 1, n))) / U[i][i]
16
17     return x
```

Fonte: Elaboração Própria

2.8. Tabelas e Gráficos

A função gerar_tabela() monta uma tabela com todas as soluções, tempo de execução e erro, salvando como imagem. Já as funções grafico_convergencia() e grafico_iteracoes() geram visualmente a evolução dos valores de x_1 , x_2 , x_3 ao longo das iterações dos métodos iterativos e também geram a comparação do número de iterações entre Jacobi e Gauss-Seidel.

Trecho 8 - Geração do gráfico de iterações

```
1  # GERAÇÃO DO GRÁFICO DE QUANTIDADE DE ITERAÇÕES DOS MÉTODOS ITERATIVOS
2  def grafico_iteracoes(it_jacobi, it_seidel):
3      plt.figure(figsize=(6, 6))
4      plt.bar(["Jacobi", "Seidel"], [it_jacobi, it_seidel])
5      plt.ylabel("Número de Iterações")
6      plt.title("Comparação do Número de Iterações")
7      plt.tight_layout()
8      plt.savefig("grafico_iteracoes.png")
9      plt.close()
```

Fonte: Elaboração Própria

Trecho 9 - Geração do gráfico de convergência

```
1 # GERAÇÃO DO GRÁFICO DE CONVERGÊNCIA DOS MÉTODOS ITERATIVOS
2 def grafico_convergencia(hist_jacobi, it_jacobi, hist_seidel, it_seidel):
3     plt.figure(figsize=(10, 6))
4
5     for i in range(hist_jacobi.shape[1]):
6         plt.plot(range(it_jacobi), hist_jacobi[:, it_jacobi, i], linestyle='--', marker='o', label=f"Jacobi x{i+1}")
7
8     for i in range(hist_seidel.shape[1]):
9         plt.plot(range(it_seidel), hist_seidel[:, it_seidel, i], linestyle='-', marker='s', label=f"Seidel x{i+1}")
10
11     plt.xlabel("Iterações")
12     plt.ylabel("Valores das Incógnitas")
13     plt.title("Convergência dos Métodos Iterativos")
14     plt.legend()
15     plt.grid(True)
16     plt.tight_layout()
17     plt.savefig("grafico_convergencia.png")
18     plt.close()
```

Fonte: Elaboração Própria

Trecho 10 - Geração da Tabela Final

```
1 # GERAÇÃO DA TABELA COM TODAS AS SOLUÇÕES DOS MÉTODOS
2 def gerar_tabela(solucoes, tempos, erros):
3     nomes = ["LU", "Gauss Clássico", "Cholesky", "Gauss-Jordan", "Jacobi", "Seidel"]
4
5     def formatar(sol):
6         if sol is None:
7             return ["N/A"] * 3
8         return [f"x{i+1} = {v:.6g}" for i, v in enumerate(sol)]
9
10    dados = []
11    for i, sol in enumerate(solucoes):
12        linha = [nomes[i]] + formatar(sol) + [f"{tempos[i]:.2e}s"]
13        linha.append(f"{erros[i]:.2e}" if erros[i] is not None else "N/A")
14        dados.append(linha)
15
16    fig = plt.figure(figsize=(12, 4))
17    grid = plt.GridSpec(1, 1)
18    ax = fig.add_subplot(grid[0])
19    ax.axis("off")
20    ax.set_title("Resumo das Soluções", fontsize=14, pad=10)
21    colunas = ["Método", "x1", "x2", "x3", "Tempo", "Erro"]
22    tabela = ax.table(cellText=dados, colLabels=colunas, loc="center", cellLoc="left")
23    tabela.set_fontsize(10)
24    tabela.scale(1.5, 2)
25    plt.tight_layout()
26    plt.savefig("tabela_solucoes.png")
27    plt.close()
```

Fonte: Elaboração Própria

2.9. Interface

Foi implementado um menu interativo de linha de comando que permite ao usuário escolher qual método de resolução deseja executar. As opções incluem a execução individual de cada método ou a execução de todos simultaneamente para realização de uma análise comparativa completa, onde gráficos de comparativos e uma tabela são gerados no formato .png.

Trecho 11 - Interface em Terminal

```
1 def main():
2     os.system('cls' if os.name == 'nt' else 'clear')
3
4     print("==== TRABALHO 3 - SISTEMAS DE EQUAÇÕES LINEARES ====")
5
6     while True:
7         mostrarSistema()
8
9         print("\nEscolha o método a ser utilizado:")
10        print("[1] Decomposição LU")
11        print("[2] Gauss-Compacto")
12        print("[3] Cholesky")
13        print("[4] Gauss-Jordan")
14        print("[5] Jacobi-Richardson")
15        print("[6] Gauss-Seidel")
16        print("[7] Comparar todos")
17        print("[8] Sair")
18        try:
19            opcao = int(input("→ "))
20            if opcao == 8:
21                print("Encerrando ... ")
22                break
23            elif 1 ≤ opcao ≤ 7:
24                executar(opcao)
25            else:
26                print("Opção inválida.")
27        except ValueError:
28            print("Digite um número válido.")
```

Fonte: Elaboração Própria

3. ANÁLISE E DISCUSSÃO

Neste tópico, será apresentado e comparado os resultados obtidos pela aplicação dos métodos diretos e iterativos no sistema linear proposto. A análise foca na precisão da solução, no tempo de execução e, para os métodos iterativos, no número de iterações. Abaixo, é possível verificar o sistema linear dado pelo enunciado do trabalho 3.

$$\begin{cases} 10x_1 + 2x_2 - x_3 = 27 \\ -3x_1 - 6x_2 + 2x_3 = -61.5 \\ x_1 + x_2 + 5x_3 = -21.5 \end{cases}$$

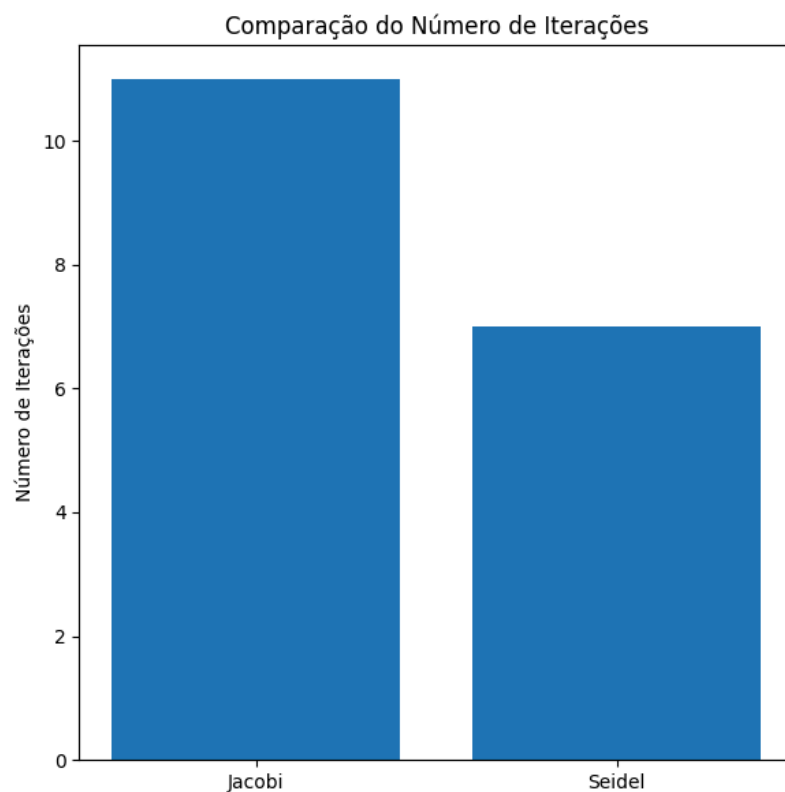
Com base nesse sistema linear, todos os métodos implementados foram executados chegando nos seguintes resultados.

Tabela 1 - Resultados obtidos

Método	x ₁	x ₂	x ₃	Tempo	Erro
LU	x1 = 0.5	x2 = 8	x3 = -6	2.28e-04s	N/A
Gauss-Compacto	x1 = 0.5	x2 = 8	x3 = -6	5.03e-05s	N/A
Cholesky	N/A	N/A	N/A	1.19e-04s	N/A
Gauss-Jordan	x1 = 0.5	x2 = 8	x3 = -6	6.01e-05s	N/A
Jacobi	x1 = 0.499997	x2 = 7.99999	x3 = -6.00001	1.68e-04s	3.10e-06
Seidel	x1 = 0.500003	x2 = 8	x3 = -6	1.19e-04s	4.05e-06

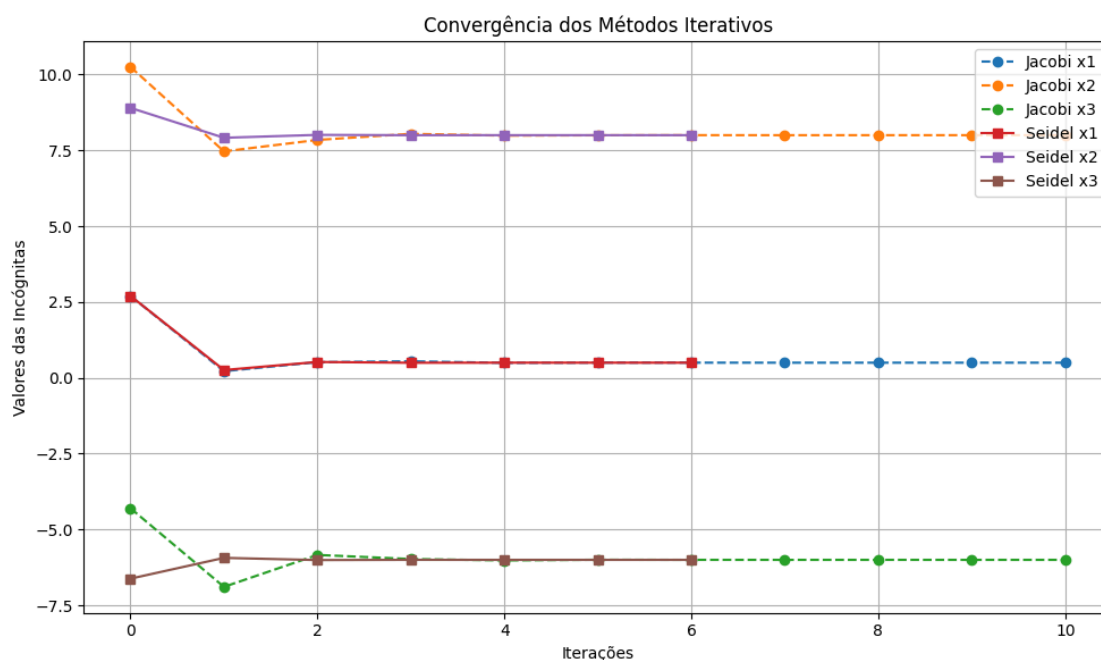
Fonte: Elaboração Própria

Gráfico 1 - Iterações por Método iterativo



Fonte: Elaboração Própria utilizando Matplotlib

Gráfico 2 - Convergência por método iterativo



Fonte: Elaboração Própria utilizando Matplotlib

Com base na tabela e nos gráficos mostrados acima, pode-se verificar que, os métodos diretos (LU, Gauss-Compacto e Gauss-Jordan) apresentaram a mesma solução exata, confirmando consistência entre eles. O método de Cholesky não pôde ser aplicado neste sistema, visto que a matriz não é simétrica.

Entre os iterativos, Jacobi e Gauss-Seidel também chegaram a soluções muito próximas, com diferenças mínimas, demonstrando boa precisão. O Jacobi-Richardson teve erro ligeiramente menor, mas levou um pouco mais de tempo que o Seidel.

Em relação ao tempo de execução, os métodos de Gauss-Compacto e de Gauss-Jordan foram os mais rápidos. Todos os métodos aplicáveis mostraram-se eficazes para o sistema resolvido, fora o de Cholesky, que dado o sistema linear do problema, não é possível utilizá-lo.

4. CONCLUSÃO

Neste trabalho, foram aplicados e comparados diferentes métodos numéricos para a resolução de um sistema linear, dado no enunciado do trabalho. Os resultados mostraram que todos os métodos diretos aplicáveis (LU, Gauss Clássico e Gauss-Jordan) forneceram a mesma solução exata, demonstrando consistência e

estabilidade entre eles.

Os métodos iterativos (Jacobi e Gauss-Seidel) também apresentaram resultados muito próximos, com erros pequenos e boa convergência, mesmo com poucas iterações. O método de Cholesky, apesar de ser um método direto, não pôde ser utilizado, pois o sistema linear fornecido possui uma matriz que não é simétrica, requisito fundamental para a aplicação dessa técnica.

Conclui-se que todos os métodos utilizados foram eficazes dentro de suas condições. A escolha entre métodos diretos ou iterativos depende das propriedades da matriz do sistema. Métodos diretos são indicados para sistemas pequenos e bem definidos, enquanto os iterativos oferecem vantagens em sistemas maiores ou com necessidade de controle sobre o erro.

A realização deste trabalho mostrou o quão importante são esses métodos de resolução de sistemas lineares na prática, ajudando a entender melhor a teoria e quando cada método funciona melhor, dependendo do tipo de sistema que é dado.